



사람의 판단은 왜 흔들리는가

확증편향부터 집단사고까지 — 공학자가 알아야 할 6가지 판단의 함정

오늘 배울 것



판단은 왜 자동으로 흔들리는가

빠르고 직관적인 사고(시스템1)가 만드는 예측 가능한 오류의 패턴을 이해한다.



정보를 다루는 세 가지 함정

확증편향 · 기준점 효과 · 가용성 편향 — 데이터를 왜곡해서 보는 습관을 짚는다.



결정을 미루는 두 가지 함정

손실회피와 과신이 어떻게 나쁜 결정을 '합리적으로 보이게' 만드는지 본다.



집단은 왜 더 나쁜 결정을 하는가

집단사고의 구조와 챌린저호 사례로, 공학 조직의 실패 메커니즘을 분석한다.

나는 합리적으로 판단하고 있을까?

아래 세 상황, 여러분도 겪어본 적 있나요? 옆 사람과 1분간 이야기해 보세요.



내 코드는 버그가 없을 것 같아

테스트를 짤 때 나도 모르게 '통과할 만한' 케이스만 먼저 떠올린 적 없나요?



최근 뉴스에 나온 사고, 왠지 나도 위험할 것 같아

확률은 낮다고 알면서도, 생생한 사고 소식을 본 뒤엔 유독 불안해합니다.



이 프로젝트, 이번엔 일정 안에 끝낼 것 같아

지난 학기 과제도 그랬지만, 이번엔 왠지 늦지 않을 것 같은 그 느낌.



공통점: 셋 다 '논리적으로 생각했다'고 믿지만, 사실은 정해진 패턴의 오류다.

인지 편향이란? — 빠른 뇌가 만드는 오류

인지 편향(cognitive bias) = 뇌가 빠르게 판단하려고 쓰는 지름길(휴리스틱)이 만드는, 예측 가능한 판단의 오류

시스템 1 · 빠른 직관

- 즉각적이고 자동적, 노력이 안 든다
- 감으로 판단 — 대부분의 일상 결정
- 예) 익숙한 코드 패턴을 한눈에 알아봄

시스템 2 · 느린 숙고

- 의식적이고 논리적, 노력이 든다
- 데이터를 분석하고 검증 — 느리지만 정확
- 예) 새 알고리즘의 시간복잡도를 직접 증명



문제는 시스템1이 항상 켜져 있다는 것 — 공학자도 예외 없이, 매일 이 지름길을 쓴다.

확증편향: 보고 싶은 것만 본다

확증편향(confirmation bias)

내 생각과 맞는 정보는 쉽게 받아들이고, 반대되는 정보는 무시하거나 깎아내리는 경향



고전 실험 · Wason 2-4-6 과제

"2, 4, 6"을 보여주고 규칙을 맞춰보라고 하면, 대부분 '2씩 증가'라는 자기 가설에 맞는 예(8,10,12)만 확인하고, 반증할 수(1,2,3도 규칙에 맞음)는 시도하지 않는다.

공학 현장에서는

디버깅할 때 '이 코드가 원인일 거야'라고 가정하면, 그 가정을 확인하는 테스트만 돌리고 다른 가능성은 건너뛰기 쉽다. 코드 리뷰에서도 내가 짠 코드는 관대하게, 남의 코드는 엄격하게 보는 경향이 나타난다.



대응: 내 가설을 확인하는 테스트가 아니라, '틀렸다면 어떻게 보일까'를 먼저 설계하라 — 반증 가능한 테스트가 진짜 검증이다.

기준점 효과: 처음 본 숫자에 발이 묶인다

기준점 효과(anchoring effect)

먼저 제시된 숫자(기준점)가, 그 숫자와 관련 없는 다음 판단에까지 영향을 미치는 현상



고전 실험 · 룰렛과 UN 국가 수

Tversky와 Kahneman(1974)은 조작된 룰렛을 돌려 나온 숫자(10 또는 65)를 먼저 보여준 뒤 'UN 회원국 중 아프리카 국가 비율'을 물었다. 룰렛과 전혀 무관한데도, 65를 본 그룹이 훨씬 높은 답을 냈다.

공학 현장에서는

프로젝트 초기에 나온 '대략 3개월'이라는 견적이, 이후 실제 작업량과 무관하게 일정 협상의 기준점이 되어버린다. 첫 벤치마크 점수, 첫 견적 금액도 같은 방식으로 이후 판단을 붙잡는다.



대응: 첫 숫자를 들었다면 '이 숫자가 없었다면 나는 뭐라고 답했을까'를 의식적으로 되물어라.

가용성 편향: 떠올리기 쉬운 게 커 보인다

가용성 편향(availability heuristic)

머릿속에 쉽게 떠오르는(생생하고 최근인) 사건일수록, 실제보다 더 자주 일어난다고 착각하는 경향



고전 실험 · 글자 'K'의 위치

Tversky와 Kahneman(1973)은 영어 단어에서 K가 첫 글자로 쓰이는 경우와 세 번째 글자로 쓰이는 경우 중 무엇이 많은지 물었다. 실제로는 세 번째가 더 많지만, 첫 글자 단어가 '떠올리기 쉬워서' 대부분 반대로 답했다.

공학 현장에서는

최근 언론에 크게 보도된 사고 유형에 안전 예산이 쏠리고, 통계적으로 더 흔하지만 덜 극적인 실패 원인은 뒷전이 된다. 최근 겪은 장애 유형만 과도하게 대비하게 되는 것도 같은 원리다.



대응: '떠오르기 쉬운 것'과 '실제로 자주 일어나는 것'을 구분하라 — 기억이 아니라 로그·통계를 보라.

손실회피: 잃는 아픔이 얻는 기쁨보다 크다

손실회피(loss aversion)

같은 크기라도, 이득의 기쁨보다 손실의 고통을 훨씬 크게 느끼는 경향 — Kahneman·Tversky의 전망이론



이론 · 손실은 약 2배 아프다

전망이론(1979)에 따르면 사람들은 손실을 이득보다 대략 2~2.5배 크게 체감한다. 그래서 '얻을 수 있는 이득'보다 '잃을 수 있는 손실'에 훨씬 민감하게 반응하며 결정을 내린다.

공학 현장에서는

더 나은 신기술이 있어도 '기존 시스템이 잘 돌아가는데 굳이?'라며 현상 유지를 택한다. 10강에서 배운 매몰비용의 함정도 이 심리와 맞닿아 있다 — 전환에서 생길 손실이, 실제 개선 효과보다 과대평가되는 것이다.



대응: '바꾸지 않아서 잃는 기회비용'도 손실로 계산에 넣어라 — 현상 유지도 하나의 선택이다.

과신: 나는 다를 거라는 근거 없는 믿음

과신(overconfidence) · 계획 오류(planning fallacy)

자신의 판단·능력·일정 추정을 실제보다 과대평가하는 경향 — 특히 일정과 비용을 체계적으로 과소추정한다



현상 · 계획 오류(planning fallacy)

지난 프로젝트들이 늦어졌다는 걸 알면서도, 이번 프로젝트의 일정은 '최선의 시나리오'로 추정하는 경향이 널리 보고되어 있다. 계획을 세울 때 과거의 지연 이력을 무시하고 이번만은 다를 거라 믿는 것이다.

공학 현장에서는

소프트웨어 개발 일정과 대형 인프라 프로젝트는 예산·기간 초과가 흔히 보고되는 대표 사례로 꼽힌다. '90% 확신한다'고 말한 예측조차 실제 적중률은 그보다 크게 낮다는 연구도 있다.



대응: 내 경험이 아니라 '비슷한 과거 프로젝트들이 실제로 얼마나 걸렸는지'(레퍼런스 클래스)로 추정하라.

집단사고: 똑똑한 사람들이 모여도 틀린다

집단사고(groupthink, Janis 1972) = 응집력 높은 집단이 '합의'를 지키려고, 반대 의견과 위험 신호를 스스로 억누르는 현상



사례 · 챌린저호 폭발(1986)

발사 전날, 부품 제작사(Thiokol)의 엔지니어들은 낮은 기온에서 O링이 제 기능을 못 할 수 있다고 경고했다. 그러나 발사 일정 압박 속에서 경영진은 '충분한 증거가 없다'며 우려를 눌렀고, 엔지니어들도 조직의 합의에 맞서길 주저했다. 다음 날 챌린저호는 발사 73초 만에 폭발했다.



핵심은 '무능'이 아니라 '구조' — 반대 의견을 낼수록 불이익을 느끼는 분위기가 문제였다.

여섯 편향, 한눈에 정리

편향	한 줄 정의	공학 현장 신호
확증편향	내 가설에 맞는 증거만 찾는다	내 코드만 관대하게 리뷰
기준점 효과	먼저 본 숫자에 판단이 묶인다	첫 견적이 협상을 지배
가용성 편향	떠올리기 쉬운 게 흔해 보인다	최근 사고 유형에 예산 쏠림
손실회피	잃는 게 얻는 것보다 아프다	레거시 시스템 교체 회피
과신·계획 오류	이번엔 다를 거라 믿는다	일정·비용 만성적 과소추정
집단사고	합의를 지키려 경고를 누른다	이의 제기가 불이익처럼 느껴짐

편향에 맞서는 4가지 습관



사전부검(pre-mortem)

결정 전에 '이 프로젝트가 실패했다고 상상하고, 왜 실패했는지'를 먼저 적어본다 — 과신과 계획 오류를 견제한다.



반대 의견 담당자 지정

회의마다 한 명을 정해 일부러 반박하게 한다 — 집단사고와 확증편향을 동시에 견제한다.



레퍼런스 클래스로 추정

내 느낌이 아니라 '비슷한 과거 사례들의 실제 결과'로 일정·비용을 추정한다 — 기준점 효과와 과신을 견제한다.



익명으로 의견 먼저 건기

리더가 의견을 말하기 전에 익명으로 각자 판단을 먼저 건는다 — 동조 압력과 집단사고를 견제한다.

생각해보기

Q1

선배가 "이 라이브러리 성능 최고야"라고 말한 뒤로, 그 라이브러리의 장점만 눈에 띄고 단점은 잘 안 보인다. 어떤 편향이고, 어떻게 검증해야 할까?

힌트: 확증편향 — 반증 가능한 벤치마크를 먼저 설계해 보자

Q2

팀이 만장일치로 '이 설계면 충분하다'고 결론 내렸는데, 사실 아무도 위험 신호를 크게 말하지 않았을 뿐이었다. 이 상황과 챌린저호 사례의 공통점은?

힌트: 집단사고 — 침묵이 곧 동의로 오해되는 구조

Q3

과거 3번의 프로젝트가 모두 예상보다 늦게 끝났는데도, 이번 프로젝트 일정은 또 '최선의 시나리오'로 잡았다. 어떤 편향이고, 무엇으로 대체해야 할까?

힌트: 계획 오류 — 과거 프로젝트들의 실제 소요 기간(레퍼런스 클래스)

다음 실습: 편향을 데이터로 확인해 보자

편향은 '느낌'이 아니라 데이터로도 드러난다. 다음 시간엔 파이썬으로 직접 확인합니다.



1. 설문으로 기준점 효과 확인

같은 반 학생 절반에게 다른 기준값을 준 뒤 답을 수집 — 두 집단 평균을 비교해 기준점 효과를 통계로 검증



2. 과제 소요시간 계획 오류 분석

예상 소요시간 vs 실제 소요시간 데이터를 모아, 우리 반의 계획 오류가 평균 몇 %인지 계산



3. 몬테카를로로 집단 의사결정 시뮬레이션

9·10강의 random 모듈로 '침묵이 동의로 오인될 확률'을 가상 회의로 시뮬레이션



준비물은 **pandas**와 **t-검정**, 그리고 **random 모듈** — 9·10강에서 배운 그대로면 충분합니다!

오늘의 핵심 정리

- ✓ 확증편향·기준점 효과·가용성 편향 — 정보를 왜곡해서 받아들이는 세 가지 함정
- ✓ 손실회피·과신 — 잘못된 결정을 '합리적으로' 느끼게 만드는 두 가지 함정
- ✓ 집단사고 — 챌린저호처럼, 똑똑한 사람들이 모여도 침묵이 동의로 둔갑한다
- ✓ 대응은 절차다: 사전부검·반대 의견 지정·레퍼런스 클래스·익명 의견 수렴



다음 시간 · 실습편 예고

설문 데이터로 기준점 효과 검증, 계획 오류 계산, 그리고 집단 의사결정 몬테카를로까지.